Frame-aware multipath QUIC for live lectures

Progress report, prepared by Kabir Bajaj for the team and course staff

We are building a frame-aware scheduler for multipath QUIC, for students who watch live lectures on a phone with two SIM cards. Its core is a **learned bandwidth predictor**: a small neural network, compiled into TQUIC, that gives a cautious estimate of each path's capacity for the next half second, which the scheduler uses to plan every video frame. Everything runs in emulation on public cellular traces and a real recorded lecture. The final comparison (8 schedulers, 5 scenarios, 5 runs each on the real lecture) is complete. On replayed cellular traces both deadline-aware schedulers miss fewer frames than every transport-only scheduler (10.6% hand-tuned and 13.5% learned, against 16–18%). The learned predictor is the most reliable capacity estimate we measured (it overestimates 11% of the time on unseen traces, against 38% for BBR), but in this first version that has not yet turned into fewer misses than the hand-tuned estimate.

Status against the plan

Week 1

29 Oct – 4 Nov

● DONE

TQUIC builds and its test suite passes. Two network namespaces are joined by two emulated paths with their own delay, rate and queue. A 10 MB multipath download used both paths under MinRTT and round-robin in 61 of 62 runs. Public cellular traces and a replay script stand in for the planned dual-SIM traces.

Gate: two paths carry traffic over emulated links. Met.

Week 2

5 – 11 Nov

● DONE

Frame sender and receiver, per-frame logs with bytes per path, ECF implemented in TQUIC, and baseline runs of MinRTT, round-robin, redundant, ECF and single-path.

Gate: baseline deadline misses measured. Met.

Week 3

12 – 18 Nov

● DONE

Deadline-aware frame scheduler and the frame-pinning ablation inside TQUIC, and trace replay working end to end.

Gate: all schedulers run end to end. Met.

Week 4

19 – 25 Nov

● EXPERIMENTS DONE

Synthetic-lecture matrix complete (7 schedulers × 5 scenarios × 5 runs). Learned predictor built, trained and checked. Final comparison on the real lecture complete (8 schedulers × 5 scenarios × 5 runs = 200 runs). Report and slides not started.

Scope change: emulation only, learned predictor at the core

- **Emulation only.** No phones or SIM cards. Path conditions come from public cellular traces (Mahimahi: AT&T LTE and T-Mobile 3G, recorded while driving) replayed in the testbed; the Chennai dual-SIM recordings are out of scope.
- **Real lecture.** The main workload is a 60 s clip of Harvard's CopyrightX Lecture 3.2 (William Fisher; CC BY 3.0, Wikimedia Commons), encoded like a live stream at 1.57 Mbit/s: 1,441 frames at 24 fps, I-frames every 2 s (median 55 KB), P-frames about 8 KB.
- **Learned predictor as the core.** The research extension in the proposal is now the main contribution; the hand-tuned scheduler becomes an ablation.

The learned bandwidth predictor

The scheduler's key input is B_p, how fast path p will deliver over the next moments. Simple estimates get this wrong in opposite ways: BBR's estimate and our hand-tuned rule (best rate of the last 3 s) are often too high, while the recent delivery rate is far too low on a path that carries little traffic. An overestimate is what makes a frame miss its deadline.

Model

`ml_features.rs`, `ml_model.rs`

An MLP (44 inputs, two hidden layers of 32, 4 outputs). Every 100 ms, per path, it reads the last second of what the sender sees: delivery rate, send rate, queueing delay, loss, plus the current congestion window, bytes in flight, min RTT and BBR's estimate. It predicts the 10th, 20th, 30th and 50th percentile of the path's mean capacity over the next 0.5 s. The scheduler uses the 10th percentile: "at least this much, 9 times out of 10". The weights are compiled into TQUIC as constants.

Training

`analysis/ml/`, `testbed/collect.sh`

60 emulated streaming sessions under 20 random network conditions drawn from six training traces (Verizon LTE ×2, T-Mobile LTE ×2, AT&T LTE 2016, Verizon EVDO), plus constant rates and random outages. None of the test traces is used. The labels are exact, because the replay log records the true capacity. Trained with the quantile (pinball) loss in numpy on 51,210 samples from 16 conditions and validated on 13,355 samples from 4 held-out conditions. The quantile the scheduler uses was chosen on 4 further training conditions.

Checks

`eval_predictor.py`

The Rust implementation inside TQUIC matches the Python model to within 0.002% over 574 live predictions. All 566 TQUIC tests pass with the model compiled in.

Prediction accuracy on the test traces (never used in training)

ESTIMATE OF NEXT-0.5 S CAPACITY	OVERESTIMATES, TRACE	MEDIAN ERROR, TRACE	OVERESTIMATES, ALL SCENARIOS	MEDIAN ERROR, ALL
model, 10th percentile (used)	**11.3%**	**49.5%**	**5.5%**	**43.7%**
model, median	46.2%	**40.2%**	39.4%	18.4%
BBR's estimate	38.2%	53.2%	8.4%	36.9%
best of last 3 s (hand-tuned rule)	47.1%	47.7%	12.2%	**5.6%**

184,612 samples (one per path every 100 ms) from all 200 real-lecture runs. On fading cellular links (trace) the model's cautious estimate overestimates capacity a third as often as BBR's and a quarter as often as the hand-tuned rule, and the model's median is the most accurate estimate there. Its 10th percentile is exceeded 11.3% of the time on trace, so it stays calibrated on traces it never saw. On constant-rate links the hand-tuned rule is already almost exact and the model is more cautious than it needs to be.

The problem we are solving

Setting

Many students in India attend online classes on a phone over mobile data. Many phones hold two SIM cards from different carriers, so the phone has two cellular links at once. Each link alone is often slow and unsteady: a few Mbit/s, delays of 50–150 ms, and short drops in coverage. Used together, the two links could keep a live lecture smooth even when one of them falters.

Multipath QUIC makes that possible at the transport layer: one QUIC connection runs over several network paths at the same time (the IETF draft is in final review). The connection has a *scheduler* that decides, for every packet, which path it goes on.

How a live lecture travels

The video encoder turns the lecture into frames, 24 per second in our recording. Every 2 seconds it sends an **I-frame**, a complete picture (about 55 KB in our lecture). The 47 frames in between are **P-frames** that only describe what changed (about 8 KB). A frame is only usable once every one of its bytes has arrived.

The player shows frames on a fixed schedule: frame *i* is due a fixed playback delay after the sender produced it. We use 300 ms. A frame that arrives later has **missed its deadline**; the picture freezes or glitches, and a late I-frame also damages the P-frames that depend on it.

What transport-only schedulers get wrong

The standard multipath schedulers see bytes, not frames. MinRTT sends each packet on the lowest-delay path that has room; round-robin alternates paths; redundant sends everything on every path; ECF waits for the fast path when that finishes the queued data sooner. None of them knows where a frame starts or ends, or when it is due. That causes two problems:

- **Frames get split across paths.** When a frame's packets go over a fast and a slow path, the frame is complete only when its last packet arrives over the slow path, so the slow path sets the frame's delay.
- **Big frames block small ones.** A large I-frame keeps one path busy for several round trips, and the P-frames queued behind it wait, even when the other path could deliver them on time.

With two fast paths this hardly matters. With two slow cellular paths there is little room to spare, and every I-frame is a deadline risk.

> `WORKED EXAMPLE: LOW_BW SCENARIO`
>
> A 45 KB I-frame on the 1.5 Mbit/s path takes 240 ms just to transmit, plus 30 ms of one-way delay: 270 ms of the 300 ms budget, with no room for the queue of P-frames already waiting on that path. In our measurements MinRTT and ECF missed the deadline on 47–57% of I-frames in this scenario, while missing under 1% of P-frames.

Questions we answer

1. How often do transport-only multipath schedulers make lecture frames miss their playback deadline when the two paths are low-bandwidth cellular links?
2. Can a scheduler that knows each frame's size and deadline do better, and which part of its design makes the difference?

The closest recent work, MoMQ (2026), is also frame-aware and also uses TQUIC, but studies Starlink combined with Wi-Fi. Two similar low-bandwidth cellular paths, the common case for students in India, have not been studied.

Our scheduler

Every frame is sent on its own QUIC stream, and the sender tells TQUIC the frame's deadline. When a new frame is ready, the scheduler:

1. Estimates the frame's finish time on each path: `T_p = (Q_p + S) / B_p + sRTT_p / 2`, where S is the frame size, Q_p the bytes already waiting for path p, B_p the path's measured bandwidth, and sRTT_p its smoothed round-trip time.
2. If a path can finish the frame within 85% of the time left before its deadline, sends the whole frame on the path that finishes first.

3. Otherwise (usually a large I-frame) splits the frame over both paths so the two parts finish at the same time.
4. Keeps each frame on the path(s) it was given, so P-frames on the other path overtake an I-frame that is still being sent, instead of queueing behind it.

For B_p we use BBR's bottleneck-bandwidth estimate (the pacing rate overestimates it by up to 25%) and remember each path's best value for 3 seconds, because a path that carries little traffic never shows its full rate. The ablation, **frame-pin**, keeps steps 1 and 4 but drops the deadline check and never splits, which isolates the value of being deadline-aware.

What we have built

Everything runs on one laptop (WSL2, Ubuntu 24.04) from scripts in our repository, with fixed random seeds and five runs per scheduler and scenario.

Multipath QUIC stack

`third_party/tquic, branch team7`

Tencent's TQUIC (Rust) v1.6.0, which implements multipath QUIC. Our changes are 9 commits, each small and documented:

- four new schedulers in TQUIC's scheduler folder: ECF, hand-tuned deadline, frame-pin and learned, plus the predictor's path sampler and the generated model;
- hooks the frame scheduler needs: an API to pass a stream's deadline, and a check, while a packet is filled, that keeps a frame's bytes on its path(s);
- a per-path log of every stream frame sent, so we can count each video frame's bytes per path;
- BBR's bandwidth estimate exposed to schedulers;
- a fix for a TQUIC bug that stalled whole connections, and a release of frames past their deadline (see Findings).

All 551 original tests and our 15 new ones pass.

Emulated network

`testbed/`

Two Linux network namespaces, student and server, joined by two virtual cables, path A and path B. On each end of each cable, `netem` adds half the round-trip delay and `tbf` limits the rate with a bottleneck queue of 100 ms. Routing by source address pins each QUIC path to its own cable. One command builds it, and it is safe to rerun after a restart.

Checked: ping measures 40.5 ms on a 40 ms path; iperf3 gets 3.56 Mbit/s of TCP data from a 4 Mbit/s path; with different delays per path, each address sees only its own path's delay.

Cellular traces and replay

`traces/`, `testbed/replay.py`

Two public traces recorded while driving (Mahimahi project): AT&T LTE as carrier 1 (5.2 Mbit/s on average, sometimes near 1) and T-Mobile 3G as carrier 2 (2.0 Mbit/s on average, no signal 12% of the time). The replay script changes each path's rate every 100 ms. Each run starts the traces at a different, seeded point, the same for every scheduler. The outage scenario cuts path B to zero for 2 s every 30 s.

Checked: iperf3 throughput follows the trace within about 1 Mbit/s each second, and drops to 0 during an outage.

Lecture workload

`workload/`

A real lecture: a 60 s clip of Harvard CopyrightX Lecture 3.2 (CC BY 3.0), encoded like a live stream (H.264, no B-frames, an I-frame every 2 s, constant quality capped at 3 Mbit/s): 1,441 frames at 1.57 Mbit/s. A synthetic lecture (slides, pointer, webcam inset; 1,800 frames at 1.56 Mbit/s) was used for development and the first comparison. We avoided strict constant-bitrate encoding because it makes every frame about the same size and hides the I-frame problem.

Streamer

`streamer/ (Rust)`

A sender (the QUIC server) and a receiver (the QUIC client, one path per SIM address). The sender writes frame *i* to a new stream at its scheduled time and passes its deadline to TQUIC. The receiver records when each frame's stream completes. All timestamps use one monotonic clock shared by both sides.

Per frame we log: size and type, send time, completion time, deadline, and bytes sent on each path, matching the log format in our proposal.

Experiments

`testbed/run_stream.sh`

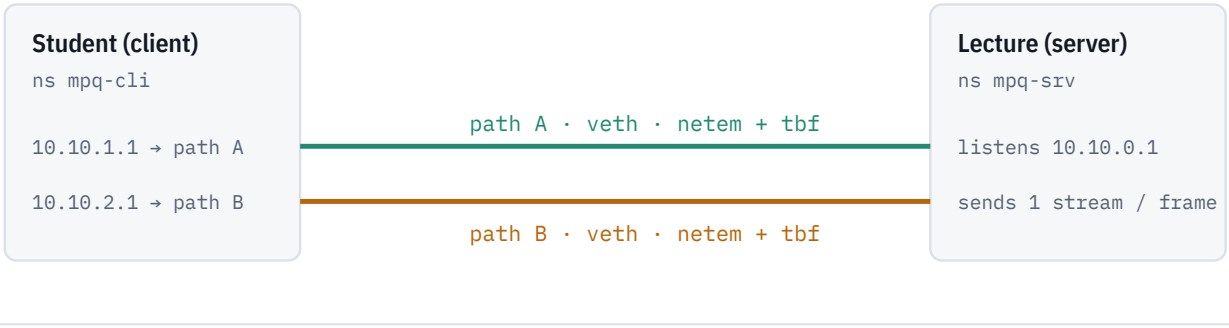
One script per scenario runs every scheduler 5 times and rebuilds the network before each run so queues start empty. The final comparison is 8 schedulers × 5 scenarios × 5 runs = 200 runs of 60 s, about 3.7 hours. It runs from a frozen copy of the programs, and each run records the exact TQUIC version it used.

Metrics and analysis

`analysis/`

Per run: frame completion time (median, P95, P99); deadline-miss rate, overall and for I- and P-frames, at playback delays of 150, 300, 500 and 1000 ms; stall count and total stall time from a player model that waits for late frames; end-to-end delay; peak reordering buffer; and share of bytes per path. Because runs with the same number see the same network, we also compare schedulers run by run. Scripts produce the tables and report figures.

Setup of the five scenarios



SCENARIO	PATH A	PATH B	WHAT IT TESTS
symmetric	4 Mbit/s, 40 ms	4 Mbit/s, 40 ms	sanity, schedulers should tie
rtt_asym	4 Mbit/s, 30 ms	4 Mbit/s, 150 ms	reordering, frame splitting
low_bw	1.5 Mbit/s, 60 ms	1 Mbit/s, 120 ms	lecture needs both paths
trace	AT&T LTE trace, 60 ms	T-Mobile 3G trace, 120 ms	real variability
outage	4 Mbit/s, 40 ms	4 Mbit/s, 0 for 2 s every 30 s	recovery after a coverage hole

Delays are round-trip. Path A is the better path in every scenario, so "single path" always uses A.

Results so far

The real lecture (main workload), all 8 schedulers. Medians over 5 runs; the best value per column is highlighted.

Frames missing a 300 ms playback deadline

SCHEDULER	SYMMETRIC	RTT_ASYM	LOW_BW	TRACE	OUTAGE
learned (ours)	**0.1%**	**0.3%**	**4.0%**	**13.5%**	**0.6%**
hand-tuned deadline	0.1%	0.1%	3.5%	**10.6%**	0.3%
frame-pin	0.2%	0.1%	3.6%	16.9%	**0.3%**
MinRTT	**0.0%**	**0.0%**	4.7%	17.8%	0.5%
ECF	**0.0%**	**0.0%**	4.9%	18.2%	0.4%
round-robin	**0.0%**	**0.0%**	**3.3%**	16.4%	0.5%
redundant	0.2%	0.1%	99.7%	52.8%	0.5%
single path (A)	0.5%	**0.0%**	99.6%	72.9%	0.3%

Total stall time (player waiting for late frames)

SCHEDULER	SYMMETRIC	RTT_ASYM	LOW_BW	TRACE	OUTAGE
learned (ours)	**34 ms**	**71 ms**	**239 ms**	**1,293 ms**	**2,662 ms**
hand-tuned deadline	53 ms	72 ms	198 ms	**1,236 ms**	1,950 ms
frame-pin	105 ms	2 ms	304 ms	1,388 ms	1,535 ms
MinRTT	**0 ms**	**0 ms**	202 ms	1,552 ms	2,721 ms
ECF	**0 ms**	**0 ms**	306 ms	1,794 ms	2,474 ms
round-robin	**0 ms**	**0 ms**	**169 ms**	1,569 ms	2,321 ms
redundant	19 ms	33 ms	13,207 ms	4,916 ms	**42 ms**
single path (A)	137 ms	**0 ms**	12,981 ms	8,023 ms	45 ms

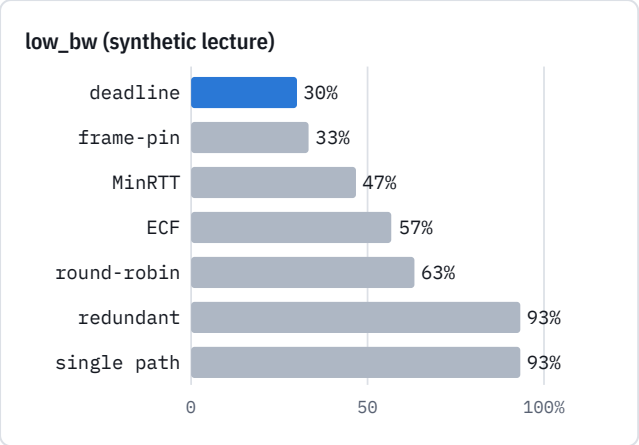
symmetric and rtt_asym: 4 Mbit/s per path is plenty for a 1.57 Mbit/s lecture, so almost nothing misses; round-robin's splitting still triples the median frame time on rtt_asym (79 ms against 28 ms). low_bw: the lecture needs both paths and its 55 KB I-frames barely fit 300 ms under any scheduler; with a 500 ms delay almost every frame arrives. outage: every multipath scheduler loses 1.5–2.7 s to frames caught on path B when it drops. One MinRTT run on rtt_asym did not deliver every frame (a rare BBR collapse) and is counted as it happened.

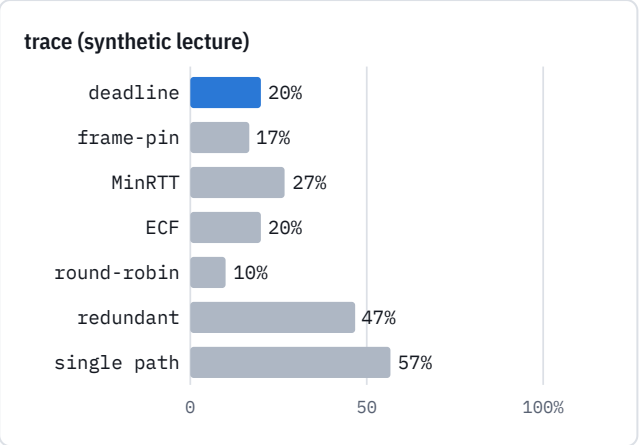
Real lecture, trace (AT&T LTE + T-Mobile 3G): where scheduling matters

SCHEDULER	MISSED @150 MS	MISSED @300 MS	MISSED @500 MS	I-FRAMES MISSED @300	MEDIAN FRAME TIME	TOTAL STALL	BYTES ON PATH B
learned (ours)	**24.6%**	**13.5%**	**5.8%**	**48.4%**	**56.4 ms**	**1,293 ms**	**33.5%**
hand-tuned deadline	23.2%	10.6%	5.1%	45.2%	64.6 ms	1,236 ms	30.9%
frame-pin	27.4%	16.9%	8.5%	67.7%	62.2 ms	1,388 ms	27.0%
MinRTT	27.6%	17.8%	8.9%	51.6%	61.8 ms	1,552 ms	33.9%
ECF	28.2%	18.2%	9.4%	48.4%	63.8 ms	1,794 ms	31.8%
round-robin	26.9%	16.4%	7.6%	54.8%	68.1 ms	1,569 ms	49.4%
redundant	57.5%	52.8%	47.0%	74.2%	410.9 ms	4,916 ms	50.3%
single path (A)	75.7%	72.9%	68.6%	77.4%	3606.5 ms	8,023 ms	0.0%

On real cellular variability the two deadline-aware schedulers miss the fewest frames at every playback delay: 10.6% (hand-tuned) and 13.5% (learned) at 300 ms, against 16–18% for MinRTT, ECF and round-robin. Run by run, the learned scheduler missed fewer frames than MinRTT and ECF in 3 of 5 runs and than round-robin in 4 of 5, and stalled less than MinRTT and ECF in 4 of 5. It did not beat the hand-tuned estimate (better in 2 of 5 runs). In 2 of the 5 runs path B had almost no signal when the connection started and no scheduler used it.

Below: the first full comparison, on the synthetic lecture, with the first build of the hand-tuned deadline scheduler (before the learned one existed and before frames past their deadline were released to any path).





Synthetic lecture, hand-tuned deadline scheduler: I-frames missing their deadline. Each run has 30 I-frames, so one I-frame is 3.3 points.

Synthetic lecture, low_bw (1.5 + 1 Mbit/s)

SCHEDULER	FRAMES MISSED	I-FRAMES MISSED	P-FRAMES MISSED	MEDIAN FRAME TIME	P95 FRAME TIME	TOTAL STALL
deadline (hand-tuned)	**0.7%**	**30.0%**	**0.3%**	**82.9 ms**	**166.7 ms**	**76 ms**
frame-pin (ablation)	0.8%	33.3%	0.2%	78.1 ms	159.5 ms	123 ms
round-robin	1.1%	63.3%	0.1%	63.1 ms	133.3 ms	85 ms
ECF	1.6%	56.7%	0.8%	93.1 ms	197.1 ms	259 ms
MinRTT	1.7%	46.7%	0.6%	91.4 ms	207.0 ms	248 ms
redundant	93.8%	93.3%	93.8%	3,765 ms	6,373 ms	6,833 ms
single path (A)	93.9%	93.3%	94.0%	3,749 ms	6,536 ms	6,818 ms

Run by run, the hand-tuned scheduler missed fewer frames and fewer I-frames than MinRTT and ECF in all 5 runs, and fewer I-frames than round-robin in all 5. The lecture does not fit on path A alone, so single-path and redundant (which sends everything on both paths and so gains no capacity) fall seconds behind.

Synthetic lecture, outage (path B drops for 2 s every 30 s)

SCHEDULER	FRAMES MISSED	I-FRAMES MISSED	TOTAL STALL	BYTES ON PATH B	FRAMES SPLIT
deadline (hand-tuned)	**0.0%**	**0.0%**	**0 ms**	**0.3%**	**0.1%**
frame-pin (ablation)	0.0%	0.0%	0 ms	0.3%	0.1%
MinRTT	0.4%	6.7%	1,856 ms	19.3%	2.8%
ECF	0.4%	6.7%	2,630 ms	20.6%	2.6%
round-robin	0.6%	6.7%	2,790 ms	45.6%	90.4%
redundant	0.0%	0.0%	0 ms	48.0%	92.3%
single path (A)	0.0%	0.0%	0 ms	0%	0%

Frames sent on path B just before an outage are stuck for up to 2 s, so MinRTT, ECF and round-robin each lost 1.9–2.8 s to stalls. The hand-tuned scheduler kept 99.7% of the bytes on the stable path, with no misses and no stalls; it beat those three in all 5 paired

runs. Path A alone can carry the lecture here, so single-path and redundant also had no misses: this scenario tests whether a scheduler avoids a flaky path.

Synthetic lecture, trace (AT&T LTE + T-Mobile 3G)

SCHEDULER	FRAMES MISSED	I-FRAMES MISSED	MEDIAN FRAME TIME	P95 FRAME TIME	TOTAL STALL	BYTES ON PATH B
deadline (hand-tuned)	**5.7%**	**20.0%**	**45.7 ms**	**325.6 ms**	**785 ms**	**17.5%**
frame-pin (ablation)	3.9%	16.7%	45.1 ms	268.1 ms	545 ms	16.7%
round-robin	3.7%	10.0%	62.9 ms	262.9 ms	673 ms	48.1%
MinRTT	3.9%	26.7%	45.7 ms	264.2 ms	1,591 ms	26.2%
ECF	4.4%	20.0%	45.6 ms	287.9 ms	1,584 ms	26.6%
redundant	41.8%	46.7%	89.8 ms	2,854 ms	3,171 ms	47.8%
single path (A)	55.7%	56.7%	1,632 ms	5,825 ms	6,063 ms	0%

On replayed cellular traces the hand-tuned scheduler does not miss fewer frames than the best baselines, but it stalls less than MinRTT in all 5 paired runs (669 ms less on average) and less than ECF in 4 of 5. Our current explanation: the scheduler remembers a path's best rate for 3 s, which helps when the slower path is idle but keeps trusting a path that has just faded. In 2 of the 5 runs path B had almost no signal when the connection started and was never used by any scheduler.

What we found

1. **Transport-only schedulers cost deadlines on cellular links.** On replayed cellular traces MinRTT, ECF and round-robin miss 16–18% of frames at a 300 ms playback delay; deadline-aware frame scheduling (hand-tuned or learned) cuts that to 10.6–13.5%. Pinning whole frames without the deadline check (frame-pin) does not help (16.9%), so the deadline check and splitting are what matter.
2. **The learned predictor is the best capacity estimate we measured where conditions change.** On unseen traces its cautious estimate overestimates 11% of the time, against 38% for BBR and 47% for the hand-tuned rule, and its median is the most accurate.
3. **Better estimates did not yet mean fewer misses than the hand-tuned scheduler.** The learned scheduler is cautious everywhere, steady links included, and moves more traffic onto the weaker path; the hand-tuned rule wins narrowly. Less caution on steady paths, and retraining with the learned scheduler's own traffic, are the next steps.
4. **On fast links the scheduler hardly matters, and on a link too slow for the lecture's I-frames no scheduler fixes it.** The playback delay does: 500 ms instead of 300 ms removes almost all misses on low_bw.

Findings along the way

A stall bug in TQUIC's multipath path selection

With path B nearly dead while it was still being validated, whole connections stalled: 15 of 1,800 frames in 90 s, while single-path on the same trace delivered everything. TQUIC kept choosing the path that needed validation frames even though its congestion window was full. Those frames are written after the congestion check, so no packet left on any path, including acknowledgements for path A. We fixed it in a few lines and plan to report it upstream.

Pinned frames and a rare BBR collapse

In one run BBR on path A fell to about one packet per second and never recovered; frames already assigned to that path never arrived (61 of 1,800). The scheduler now lets any frame that is past its deadline use either path, and the rerun scenario had no misses or stalls.

A real lecture at full quality overloads low_bw

Encoded at the same quality as our first tests, the real lecture needs 2.13 Mbit/s, about 90% of what both low_bw paths carry; queues grow without bound and every scheduler missed about 65% of frames. A live platform lowers the bitrate on such a connection, so the evaluation uses 1.57 Mbit/s.

WSL2 steps the system clock

The wall clock jumped forward by about 0.5 s every 30–40 s, which showed up as frames up to 1.1 s late. All timing now uses the monotonic clock. Week 1 transfer times taken before the fix can be about 0.5 s long; byte shares are unaffected.

Transport-only schedulers split frames even when one path is enough

In a 10 MB transfer on the 30 ms / 150 ms scenario, MinRTT still sent 48% of the bytes over the 150 ms path, because both congestion windows stay full. In the streaming runs round-robin split 89–97% of frames across both paths. This is the behaviour the frame-aware scheduler is meant to avoid.

Limitations of the current results

- All results are from emulation on one laptop (network namespaces on WSL2) with public US cellular traces; there is no live cellular run.
- The learned predictor is trained on six public traces; how well it transfers to Indian networks is untested.
- Our TQUIC changes include two bug fixes that apply to every scheduler; each comparison runs all schedulers from one frozen build.

Next steps

- Make the learned scheduler less cautious on steady paths (e.g. pick the quantile from the predicted spread), and retrain with traffic from the learned scheduler itself.
- Add figures (miss rate against playback delay, completion-time distributions).
- Write the final report and slides.